
GramVest

SIH 2026 • PS 26091

M2 - Backend, Database & API Contracts

Module implementation handbook - backend owner

TEAM IMPLEMENTATION HANDBOOK

Prepared as a practical engineering guide for independent module development, stable integration, testing, and handoff.

Source basis: SIH 26091 Implementation Blueprint and the corresponding GramVest module
Markdown handbook.

Contents

1 1. Mission	1
2 2. Why this module exists	1
3 3. Responsibilities	1
3.1 Core	1
3.2 Integration	1
4 4. Recommended tools	1
5 5. Repository	2
6 6. Database domains	2
7 7. Minimum shared fields	3
8 8. Core profile data	3
9 9. Location data	3
10 10. API contract rule	3
11 11. Endpoints	4
11.1 Profile	4
11.2 Location	4
11.3 Market	4
11.4 Opportunity	4
11.5 Finance	4
11.6 Schemes	4
11.7 Advisor	4
11.8 Reports	5
11.9 Simulator	5
12 12. Request/response design example	5
12.1 Finance request	5
12.2 Finance response	5
13 13. Validation rules	5
14 14. Database versioning	6
15 15. Environment configuration	6
16 16. Security rules	6
17 17. Health and observability	6
18 18. Seed data	6
19 19. Integration workflow	7
19.1 Step 1	7
19.2 Step 2	7
19.3 Step 3	7
19.4 Step 4	7
19.5 Step 5	7
19.6 Step 6	7

19.7 Step 7	7
20 20. Definition of done	7
21 21. Common mistakes	7

1. Mission

Build the central backend that all other modules talk to. M2 owns the FastAPI application, database model, request/response validation, authentication/session plumbing if needed, common schemas, error handling, logging, and integration contracts.

The blueprint recommends Python + FastAPI, PostgreSQL, PostGIS, and pgvector. PostGIS/pgvector are used by M3/M5 respectively, but M2 owns the base database and API conventions.

2. Why this module exists

Without a central contract, every developer will build incompatible objects. M2 creates the single source of truth so that:

- M3 knows how to return market data
- M4 knows how to return financial data
- M5 knows how to consume structured context
- M1 knows exactly what each API returns

The blueprint explicitly asks for common API contracts and seed data during foundation.

3. Responsibilities

Core

- FastAPI app
- database connection
- ORM/data-access layer
- Pydantic schemas
- common API response format
- validation
- error handling
- environment management
- CORS/auth basics as needed
- logging/audit fields

Integration

- route module outputs into one consistent API
- expose health/status endpoints
- provide seed/demo endpoints or scripts
- maintain OpenAPI documentation generated by FastAPI

4. Recommended tools

Task	Tool
Backend	Python + FastAPI
Validation	Pydantic
DB	PostgreSQL
DB admin	DBeaver or pgAdmin
API testing	Postman/Insomnia
Python env	venv/uv/poetry; choose one and standardize
ORM/data access	SQLAlchemy or agreed alternative
Migrations	Alembic
Containerization	Docker

Task	Tool
Version control	Git + GitHub

Use one ORM strategy. Do not allow every member to write raw SQL in random places.

5. Repository

```
backend/  
├── api/  
│   ├── routes/  
│   └── dependencies.py  
├── models/  
├── schemas/  
├── repositories/  
├── services/  
├── core/  
├── db/  
├── tests/  
└── main.py
```

6. Database domains

The blueprint lists these logical data domains:

- user/profile
- geography
- demographics
- businesses
- markets
- prices
- schemes
- financial scenarios
- knowledge base

Suggested relational design:

```
users  
profiles  
locations  
business_categories  
businesses  
markets  
prices  
schemes  
scheme_versions  
scheme_eligibility_rules  
financial_scenarios  
financial_results  
opportunity_results  
risk_results  
viability_results  
advisor_sessions  
reports  
report_versions  
knowledge_documents  
knowledge_chunks
```

7. Minimum shared fields

Every important model should consider:

```
id
created_at
updated_at
source
source_date
version
confidence
status
```

Not every table needs every field; use only where meaningful.

8. Core profile data

Profile

- name
- phone/email if required by auth
- language
- business_idea
- business_category
- experience_level
- own_capital
- target_customers (optional)
- existing_assets (optional)
- preferred_scale (optional)
- desired_income (optional)
- risk_preference (optional)

9. Location data

Location

- village/town
- block
- district
- state
- latitude
- longitude
- analysis_radius_km
- source/confidence

10. API contract rule

Every API should return one consistent envelope.

Suggested:

```
{
  "success": true,
  "data": {},
  "meta": {
    "source": [],
    "confidence": 0.84,
    "assumptions": [],
    "version": "1.0"
  },
  "error": null
}
```

Failure:

```
{
  "success": false,
  "data": null,
  "meta": {},
  "error": {
    "code": "LOCATION_NOT_FOUND",
    "message": "We could not resolve the location."
  }
}
```

11. Endpoints

Profile

POST /api/profile

Create/update profile.

Location

GET /api/location/resolve

Resolve village/block/district/coordinates.

Market

POST /api/market/analyze

Run market analysis.

GET /api/market/competitors

Return mapped businesses.

Opportunity

POST /api/opportunity/score

Return opportunity indicators and evidence.

Finance

POST /api/finance/project

Calculate project financing.

POST /api/finance/repayment

Return amortization/cash flow.

POST /api/finance/dscr

Return DSCR and health indicators.

Schemes

POST /api/schemes/route

Apply versioned scheme rules.

Advisor

POST /api/advisor/chat

Return grounded AI response.

Reports

POST /api/report/generate

Generate feasibility/DPR report.

Simulator

POST /api/simulator/what-if

Recalculate scenario.

These endpoints follow the blueprint's API plan.

12. Request/response design example

Finance request

```
{
  "project_cost": 500000,
  "own_capital": 300000,
  "loan_amount": 200000,
  "annual_interest_rate": 8.0,
  "tenure_months": 84,
  "moratorium_months": 0,
  "monthly_revenue": 85000,
  "monthly_expenses": 63000
}
```

Finance response

```
{
  "project_cost": 500000,
  "own_contribution": 300000,
  "financing_required": 200000,
  "emi": 3119,
  "monthly_revenue": 85000,
  "monthly_expenses": 63000,
  "monthly_surplus": 22000,
  "dscr": 1.51,
  "break_even_month": 11,
  "assumptions": []
}
```

Use actual calculated outputs from M4; the example is only a contract illustration.

13. Validation rules

Validate server-side:

- positive project cost
- non-negative own capital
- sensible loan amount
- supported category
- valid coordinates
- valid radius
- valid dates/versions
- scenario bounds

Return friendly backend error codes for M1 to translate.

14. Database versioning

Use migrations.

Every schema change should be:

```
migration
→ test
→ review
→ merge
```

Never manually edit production/demo database structure without a migration.

15. Environment configuration

Use environment variables for:

```
DATABASE_URL
LLM/API keys
MAP/config keys if required
OBJECT_STORAGE if used
APP_ENV
```

Never commit secrets.

16. Security rules

- validate API inputs
- never expose secrets in frontend
- separate user profile data from public market data
- log assumptions/version where traceability matters
- avoid collecting unnecessary sensitive identity data
- use CORS carefully
- use parameterized/ORM queries

The blueprint explicitly calls out server-side validation, secret protection, data separation and traceability.

17. Health and observability

Create at minimum:

```
GET /health
```

Return:

- API status
- DB status
- version

For prototype logging, capture:

- endpoint
- timestamp
- request id
- error

Do not log sensitive user details unnecessarily.

18. Seed data

M2 owns the mechanism for loading demo data.

Example:

```
python scripts/seed_demo.py
```

Seed:

- demo entrepreneur
- demo business
- demo location
- sample competitors
- market records
- sample prices
- scheme versions

M3 owns the actual market dataset content; M4 owns financial demo assumptions; M5 owns knowledge documents.

19. Integration workflow

Step 1

Freeze schemas with team.

Step 2

Create DB tables/migrations.

Step 3

Create Pydantic request/response models.

Step 4

Create endpoints returning mock/seed values.

Step 5

M1 connects to contracts.

Step 6

Replace mock service implementations with M3/M4/M5 outputs.

Step 7

Add integration tests.

20. Definition of done

M2 is done when:

- database schema is committed
- migrations work from an empty DB
- every shared endpoint has a schema
- OpenAPI docs are available
- error format is standardized
- seed data loads
- unit/API tests pass
- M1 can integrate without reading M3/M4 internal code

21. Common mistakes

- changing API field names without team agreement

- mixing business logic into routes
- hard-coding scheme numbers in route code
- returning free-form dictionaries from every endpoint
- leaking stack traces to frontend
- putting LLM calculation into backend finance routes
- storing secrets in Git